

Knowledge-Base Search Engine (RAG)

Features:

- Document Ingestion: PDF processing and text chunking.
- Vector Embeddings: Local text vectorization using nomic-embed-text
- RAG Pipeline: Context-aware retrieval using ChromaDB.
- Synthesis: Succinct answer generation via Llama3.

Tech Stack:

- Backend: FastAPI (Python)
- Orchestration: LangChain
- Vector Database: ChromaDB
- LLM Engine: Ollama (Llama 3 & Nomic-Embed-Text)

Prerequisites:

- Python 3.10+
- [Ollama](#) installed and running.

Prepare the AI Models:

In terminal type the necessary models

- `ollama pull llama3`
- `ollama pull nomic-embed-text`

Installation:

- Install required Python packages
`pip install fastapi uvicorn langchain langchain-community langchain-ollama pypdf chromadb python-multipart`

Running the Application:

- Start the FastAPI server:
`python -m uvicorn main:app --reload`
The server will be available at <http://127.0.0.1:8000/docs>

Upload Document: Use the /upload endpoint.

Select a PDF file

Click **Execute**. We should receive a Success message.

Ask Questions: Use the /ask endpoint.

Architecture:

The system follows a standard RAG workflow:

1. **Ingestion:** PDF text is extracted and split into 500-character chunks with overlap.
2. **Embedding:** Chunks are converted into vectors and stored in a local ChromaDB instance.
3. **Retrieval:** User queries are vectorized to find the most relevant document chunks.
4. **Synthesis:** The LLM (Llama 3) uses the retrieved chunks as context to answer the query succinctly.